NAME:	**M.MANASA**
REG NO:	**192425189**
COURSE CODE:	**CSA4315**
COURSE:	**INTERNET PROGRAMMING**

EXP-9

9. TechSavvy, a leading retailer of electronic gadgets, has launched an online shopping platform to provide customers with a seamless and convenient shopping experience. The website features a wide range of products, including smartphones, laptops, tablets, and accessories, all categorized for easy browsing. Key functionalities include user registration and login, product search and filtering, detailed product descriptions and reviews, secure payment options, and order tracking. Customers can add items to their cart, create wish lists, and receive personalized recommendations based on their browsing and purchase history. The platform also offers exclusive online discounts and promotions, ensuring customers get the best deals on the latest gadgets.The core of an online shop is the product catalogue and the cart. The users browse products which are grouped in categories and add the desired ones to the cart. Next, there is transaction handling, which in its simplest form consists in the order being sent via e-mail to the seller, but most often INTERNET PROGRAMMING LAB EXPRIMENTS it takes the whole buying process through separate order statuses.

HTML CODE:

```
import os
import secrets
from functools import wraps
from datetime import datetime
from flask import (
  Flask, render_template, request, redirect, url_for,
  session, flash, jsonify, abort
)
from werkzeug.security import generate_password_hash, check_password_hash
from db import get_db, init_db
```

```
app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'shopease-super-secret-key-college-demo-2026')

# Initialize DB if not already initialized
with app.app_context():
    init_db()

CATEGORIES = [
    "Fashion",
    "Electronics",
    "Accessories",
    "Home and Kitchen",
    "Beauty and Personal Care",
    "Books",
    "Sports and Fitness"
]

# ------------------------------------------------------------
# Jinja Custom Filters & Context Processors
# ------------------------------------------------------------
@app.template_filter('inr')
def format_inr(value):
    """Format numbers into Indian Rupee format, e.g. 1,499.00"""
    try:
        val = float(value)
        return f"₹{val:,.2f}"
    except (ValueError, TypeError):
        return f"₹{value}"

@app.context_processor
```

```
def inject_global_data():
    """Make common variables available across all templates."""
    cart = session.get('cart', {})
    cart_count = sum(cart.values()) if isinstance(cart, dict) else 0

    # Ensure CSRF token exists in session
    if 'csrf_token' not in session:
        session['csrf_token'] = secrets.token_hex(16)

    return {
        'cart_count': cart_count,
        'all_categories': CATEGORIES,
        'current_year': datetime.now().year,
        'csrf_token': session.get('csrf_token')
    }

# ---------------------------------------------------------------
# Auth & Security Helpers
# ---------------------------------------------------------------
def verify_csrf():
    """Validates the CSRF token from form or header."""
    token = request.form.get('csrf_token') or request.headers.get('X-CSRFToken')
    if not token or token != session.get('csrf_token'):
        abort(400, description="Invalid or missing CSRF token.")

def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash("Please log in to access this page.", "warning")
            return redirect(url_for('login', next=request.path))
```

```
        return f(*args, **kwargs)
    return decorated_function

def admin_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash("Admin login required.", "warning")
            return redirect(url_for('login', next=request.path))
        if not session.get('is_admin'):
            flash("Access denied. Admin privileges required.", "danger")
            return redirect(url_for('home'))
        return f(*args, **kwargs)
    return decorated_function

def get_cart_details():
    """
    Recalculates cart items and totals from the database to guarantee
    pricing integrity.
    """
    cart = session.get('cart', {})
    if not cart:
        return {'items': [], 'subtotal': 0.0, 'shipping': 0.0, 'total': 0.0}

    conn = get_db()
    cursor = conn.cursor()

    items = []
    subtotal = 0.0

    # Fetch fresh product info for all IDs in session cart
```

```
product_ids = [int(pid) for pid in cart.keys() if pid.isdigit()]
if product_ids:
    placeholders = ','.join(['?'] * len(product_ids))
    cursor.execute(f"SELECT * FROM products WHERE id IN ({placeholders})", product_ids)
    products_map = {p['id']: p for p in cursor.fetchall()}

    for pid_str, qty in list(cart.items()):
        try:
            pid = int(pid_str)
            qty = max(1, int(qty))
        except (ValueError, TypeError):
            continue

        if pid in products_map:
            prod = products_map[pid]
            item_subtotal = float(prod['price']) * qty
            subtotal += item_subtotal
            items.append({
                'product_id': prod['id'],
                'name': prod['name'],
                'price': float(prod['price']),
                'category': prod['category'],
                'image_url': prod['image_url'],
                'stock': prod['stock'],
                'quantity': qty,
                'subtotal': item_subtotal
            })
        else:
            # Remove stale product from cart
            cart.pop(pid_str, None)
    session['cart'] = cart
```

```
        session.modified = True

    conn.close()

    # Free shipping on orders of ₹1,000 or more, else ₹99 flat delivery fee
    shipping = 0.0 if (subtotal >= 1000.0 or subtotal == 0.0) else 99.0
    total = subtotal + shipping

    return {
        'items': items,
        'cart_items': items,
        'subtotal': subtotal,
        'shipping': shipping,
        'total': total
    }

# ------------------------------------------------------------
# Main Customer Routes
# ------------------------------------------------------------
@app.route('/home')
@login_required
def home():
    query = request.args.get('q', '').strip()
    category = request.args.get('category', '').strip()

    conn = get_db()
    cursor = conn.cursor()

    sql = "SELECT * FROM products WHERE 1=1"
    params = []
```

```
    if query:
        sql += " AND (name LIKE ? OR description LIKE ?)"
        params.extend([f"%{query}%", f"%{query}%"])

    if category and category != 'All':
        sql += " AND category = ?"
        params.append(category)

    sql += " ORDER BY id DESC"
    cursor.execute(sql, params)
    products = cursor.fetchall()

    # Featured, Popular, New sections for main showcase
    cursor.execute("SELECT * FROM products WHERE is_featured = 1 ORDER BY rating DESC LIMIT 4")
    featured_products = cursor.fetchall()

    cursor.execute("SELECT * FROM products WHERE is_popular = 1 ORDER BY reviews_count DESC LIMIT 4")
    popular_products = cursor.fetchall()

    cursor.execute("SELECT * FROM products WHERE is_new = 1 ORDER BY id DESC LIMIT 4")
    new_products = cursor.fetchall()

    conn.close()

    return render_template(
        'index.html',
        products=products,
        featured_products=featured_products,
        popular_products=popular_products,
        new_products=new_products,
```

```
        selected_category=category,
        search_query=query
    )

@app.route('/product/<int:product_id>')
@login_required
def product_detail(product_id):
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM products WHERE id = ?", (product_id,))
    product = cursor.fetchone()

    if not product:
        conn.close()
        abort(404, description="Product not found.")

    # Fetch related products from same category
    cursor.execute(
        "SELECT * FROM products WHERE category = ? AND id != ? LIMIT 4",
        (product['category'], product_id)
    )
    related_products = cursor.fetchall()
    conn.close()

    return render_template('product.html', product=product,
related_products=related_products)

# ------------------------------------------------------------
# Cart Routes
# ------------------------------------------------------------
```

```
@app.route('/cart')
@login_required
def view_cart():
    cart_data = get_cart_details()
    return render_template('cart.html', cart=cart_data)

@app.route('/cart/add', methods=['POST'])
@login_required
def add_to_cart():
    verify_csrf()
    product_id = request.form.get('product_id')
    try:
        quantity = int(request.form.get('quantity', 1))
        if quantity < 1:
            quantity = 1
    except ValueError:
        quantity = 1

    if not product_id or not product_id.isdigit():
        flash("Invalid product specified.", "danger")
        return redirect(url_for('view_cart'))

    # Verify product exists & check stock
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT id, name, stock FROM products WHERE id = ?", (int(product_id),))
    product = cursor.fetchone()
    conn.close()

    if not product:
        flash("Product not found.", "danger")
```

```
    return redirect(url_for('home'))

cart = session.get('cart', {})
current_qty = cart.get(str(product_id), 0)
new_qty = current_qty + quantity

if new_qty > product['stock']:
    new_qty = product['stock']
    flash(f"Only {product['stock']} units available in stock.", "warning")
else:
    flash(f"Added '{product['name']}' to your cart!", "success")

cart[str(product_id)] = new_qty
session['cart'] = cart
session.modified = True

# If action was "buy_now", redirect straight to checkout
if request.form.get('buy_now') == '1':
    return redirect(url_for('checkout'))

# Check if AJAX request
if request.headers.get('X-Requested-With') == 'XMLHttpRequest':
    cart_data = get_cart_details()
    return jsonify({
        'status': 'success',
        'cart_count': sum(cart.values()),
        'message': f"Added '{product['name']}' to cart."
    })

return redirect(request.referrer or url_for('view_cart'))
```

```
@app.route('/cart/update', methods=['POST'])
def update_cart():
    verify_csrf()
    product_id = request.form.get('product_id')
    try:
        quantity = int(request.form.get('quantity', 1))
    except ValueError:
        quantity = 1

    cart = session.get('cart', {})
    if product_id in cart:
        if quantity <= 0:
            cart.pop(product_id, None)
            flash("Item removed from cart.", "info")
        else:
            # Check stock limit
            conn = get_db()
            cursor = conn.cursor()
            cursor.execute("SELECT stock, name FROM products WHERE id = ?",
(int(product_id),))
            prod = cursor.fetchone()
            conn.close()

            if prod and quantity > prod['stock']:
                cart[product_id] = prod['stock']
                flash(f"Only {prod['stock']} units available for '{prod['name']}'.", "warning")
            else:
                cart[product_id] = quantity
                flash("Cart updated successfully.", "success")

        session['cart'] = cart
```

```
        session.modified = True

    if request.headers.get('X-Requested-With') == 'XMLHttpRequest':
        cart_data = get_cart_details()
        return jsonify({
            'status': 'success',
            'cart_count': sum(cart.values()),
            'cart': cart_data
        })

    return redirect(url_for('view_cart'))

@app.route('/cart/remove', methods=['POST'])
def remove_from_cart():
    verify_csrf()
    product_id = request.form.get('product_id')
    cart = session.get('cart', {})

    if product_id in cart:
        del cart[product_id]
        session['cart'] = cart
        session.modified = True
        flash("Item removed from cart.", "info")

    return redirect(url_for('view_cart'))

@app.route('/cart/clear', methods=['POST'])
def clear_cart():
    verify_csrf()
    session['cart'] = {}
    session.modified = True
```

```
    flash("Your shopping cart has been cleared.", "info")
    return redirect(url_for('view_cart'))

# ------------------------------------------------------------
# Checkout & Order Placement
# ------------------------------------------------------------
@app.route('/checkout', methods=['GET', 'POST'])
@login_required
def checkout():
    cart_data = get_cart_details()

    if not cart_data['items']:
        flash("Your cart is empty. Add products before proceeding to checkout.", "warning")
        return redirect(url_for('home'))

    if request.method == 'POST':
        verify_csrf()

        full_name = request.form.get('full_name', '').strip()
        email = request.form.get('email', '').strip()
        phone = request.form.get('phone', '').strip()
        address = request.form.get('address', '').strip()
        city = request.form.get('city', '').strip()
        state = request.form.get('state', '').strip()
        postal_code = request.form.get('postal_code', '').strip()
        payment_method = request.form.get('payment_method', 'Cash on Delivery')

        # Server-side validation
        if not all([full_name, email, phone, address, city, state, postal_code]):
            flash("All shipping fields are required.", "danger")
            return render_template('checkout.html', cart=cart_data)
```

```
    # Generate a unique professional order number
    order_number = f"ORD-{datetime.now().strftime('%Y%m%d')}-
{secrets.randbelow(8999) + 1000}"

    conn = get_db()
    cursor = conn.cursor()

    try:
      # 1. Insert Order
      cursor.execute('''
        INSERT INTO orders (
          order_number, user_id, full_name, email, phone,
          address, city, state, postal_code, payment_method,
          subtotal, shipping_fee, total_amount, status
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
      ''', (
        order_number, session['user_id'], full_name, email, phone,
        address, city, state, postal_code, payment_method,
        cart_data['subtotal'], cart_data['shipping'], cart_data['total'],
        'Processing'
      ))

      order_id = cursor.lastrowid

      # 2. Insert Order Items & adjust stock
      for item in cart_data['items']:
        cursor.execute('''
          INSERT INTO order_items (
            order_id, product_id, product_name, product_image,
            price_at_purchase, quantity, subtotal
```

```
            ) VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (
            order_id, item['product_id'], item['name'], item['image_url'],
            item['price'], item['quantity'], item['subtotal']
        ))

        # Decrement product stock safely
        cursor.execute('''
            UPDATE products
            SET stock = MAX(0, stock - ?)
            WHERE id = ?
        ''', (item['quantity'], item['product_id']))

    conn.commit()

    # 3. Clear shopping cart
    session['cart'] = {}
    session.modified = True

    flash(f"Order #{order_number} placed successfully!", "success")
    return redirect(url_for('order_confirmation', order_number=order_number))

  except Exception as e:
    conn.rollback()
    flash("An error occurred while processing your order. Please try again.", "danger")
    return render_template('checkout.html', cart=cart_data)
  finally:
    conn.close()

# Pre-populate customer name/email from session if available
conn = get_db()
```

```
    cursor = conn.cursor()
    cursor.execute("SELECT name, email FROM users WHERE id = ?", (session['user_id'],))
    user_row = cursor.fetchone()
    conn.close()

    return render_template('checkout.html', cart=cart_data, user=user_row)

@app.route('/order-confirmation/<order_number>')
@login_required
def order_confirmation(order_number):
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute('''
        SELECT * FROM orders
        WHERE order_number = ? AND (user_id = ? OR ?)
    ''', (order_number, session['user_id'], 1 if session.get('is_admin') else 0))
    order = cursor.fetchone()

    if not order:
        conn.close()
        abort(404, description="Order not found.")

    cursor.execute("SELECT * FROM order_items WHERE order_id = ?", (order['id'],))
    items = cursor.fetchall()
    conn.close()

    return render_template('order_confirmation.html', order=order, items=items)

@app.route('/orders')
@login_required
```

```
def orders():
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute('''
        SELECT * FROM orders
        WHERE user_id = ?
        ORDER BY id DESC
    ''', (session['user_id'],))
    orders_list = cursor.fetchall()

    orders_with_items = []
    for ord_row in orders_list:
        cursor.execute("SELECT * FROM order_items WHERE order_id = ?", (ord_row['id'],))
        items = cursor.fetchall()
        orders_with_items.append({
            'order': ord_row,
            'order_items': items,
            'items': items
        })

    conn.close()
    return render_template('orders.html', orders=orders_with_items)

# ------------------------------------------------------------
# User Authentication Routes
# ------------------------------------------------------------
@app.route('/', methods=['GET', 'POST'])
@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'user_id' in session:
```

```
    if session.get('is_admin'):
        return redirect(url_for('admin_dashboard'))
    return redirect(url_for('home'))

  if request.method == 'POST':
    verify_csrf()
    email = request.form.get('email', '').strip().lower()
    password = request.form.get('password', '')
    next_page = request.form.get('next') or request.args.get('next')

    if not email or not password:
        flash("Please enter both email and password.", "warning")
        return render_template('login.html', next=next_page)

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE email = ?", (email,))
    user = cursor.fetchone()
    conn.close()

    if user and check_password_hash(user['password_hash'], password):
        session['user_id'] = user['id']
        session['user_name'] = user['name']
        session['user_email'] = user['email']
        session['is_admin'] = bool(user['is_admin'])
        flash(f"Welcome back, {user['name']}!", "success")

        if next_page and not next_page.startswith('//') and not next_page.startswith('http')
and next_page not in ['/', '/login']:
            return redirect(next_page)
```

```
            if user['is_admin']:
                return redirect(url_for('admin_dashboard'))
            return redirect(url_for('home'))
        else:
            flash("Invalid email or password. Please try again.", "danger")

    return render_template('login.html', next=request.args.get('next'))

@app.route('/register', methods=['GET', 'POST'])
def register():
    if 'user_id' in session:
        return redirect(url_for('home'))

    if request.method == 'POST':
        verify_csrf()
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip().lower()
        password = request.form.get('password', '')
        confirm_password = request.form.get('confirm_password', '')

        if not name or not email or not password:
            flash("All fields are required.", "warning")
            return render_template('register.html')

        if len(password) < 6:
            flash("Password must be at least 6 characters long.", "warning")
            return render_template('register.html')

        if password != confirm_password:
            flash("Passwords do not match.", "danger")
            return render_template('register.html')
```

```
    conn = get_db()
    cursor = conn.cursor()

    try:
        cursor.execute("SELECT id FROM users WHERE email = ?", (email,))
        if cursor.fetchone():
            flash("An account with this email already exists. Please log in.", "warning")
            conn.close()
            return redirect(url_for('login'))

        hashed_pw = generate_password_hash(password)
        cursor.execute(
            "INSERT INTO users (name, email, password_hash, is_admin) VALUES (?, ?, ?, 0)",
            (name, email, hashed_pw)
        )
        conn.commit()

        flash("Account created successfully! Please log in with your credentials to start
shopping.", "success")
        return redirect(url_for('login'))

    except Exception as e:
        conn.rollback()
        flash("Error creating account. Please try again.", "danger")
        return render_template('register.html')
    finally:
        conn.close()

  return render_template('register.html')
```

```
@app.route('/logout')
def logout():
    session.pop('user_id', None)
    session.pop('user_name', None)
    session.pop('user_email', None)
    session.pop('is_admin', None)
    session.pop('cart', None)
    flash("You have been logged out successfully.", "info")
    return redirect(url_for('login'))

# ------------------------------------------------------------
# Admin Dashboard & Product Management Routes
# ------------------------------------------------------------
@app.route('/admin')
@admin_required
def admin_dashboard():
    conn = get_db()
    cursor = conn.cursor()

    # Sales Statistics
    cursor.execute("SELECT COUNT(*) FROM orders")
    total_orders = cursor.fetchone()[0]

    cursor.execute("SELECT COALESCE(SUM(total_amount), 0) FROM orders")
    total_revenue = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM products")
    total_products = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM users WHERE is_admin = 0")
    total_customers = cursor.fetchone()[0]
```

```
    # All Products for Inventory Table
    cursor.execute("SELECT * FROM products ORDER BY id DESC")
    products = [dict(p) for p in cursor.fetchall()]

    # All Orders for Order Management
    cursor.execute('''
        SELECT o.*, u.name as customer_name
        FROM orders o
        JOIN users u ON o.user_id = u.id
        ORDER BY o.id DESC
    ''')
    orders = cursor.fetchall()

    conn.close()

    stats = {
        'total_orders': total_orders,
        'total_revenue': total_revenue,
        'total_products': total_products,
        'total_customers': total_customers
    }

    return render_template('admin.html', stats=stats, products=products, orders=orders)

@app.route('/admin/products/add', methods=['POST'])
@admin_required
def admin_add_product():
    verify_csrf()
    name = request.form.get('name', '').strip()
    category = request.form.get('category', '').strip()
```

```
    try:
        price = float(request.form.get('price', 0))
        stock = int(request.form.get('stock', 10))
    except ValueError:
        flash("Invalid price or stock value.", "danger")
        return redirect(url_for('admin_dashboard'))

    description = request.form.get('description', '').strip()
    image_url = request.form.get('image_url', '').strip()
    is_featured = 1 if request.form.get('is_featured') else 0
    is_popular = 1 if request.form.get('is_popular') else 0
    is_new = 1 if request.form.get('is_new') else 0

    if not name or not category or price <= 0:
        flash("Please provide product name, valid category, and price.", "danger")
        return redirect(url_for('admin_dashboard'))

    if not image_url:
        image_url = "https://images.unsplash.com/photo-1526170375885-
4d8ecf77b99f?w=600&auto=format&fit=crop&q=80"

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO products (
            name, category, price, stock, description, image_url,
            is_featured, is_popular, is_new
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    ''', (name, category, price, stock, description, image_url, is_featured, is_popular, is_new))
    conn.commit()
    conn.close()
```

```
    flash(f"Product '{name}' added successfully!", "success")
    return redirect(url_for('admin_dashboard'))

@app.route('/admin/products/edit/<int:product_id>', methods=['POST'])
@admin_required
def admin_edit_product(product_id):
    verify_csrf()
    name = request.form.get('name', '').strip()
    category = request.form.get('category', '').strip()
    try:
        price = float(request.form.get('price', 0))
        stock = int(request.form.get('stock', 0))
    except ValueError:
        flash("Invalid price or stock.", "danger")
        return redirect(url_for('admin_dashboard'))

    description = request.form.get('description', '').strip()
    image_url = request.form.get('image_url', '').strip()
    is_featured = 1 if request.form.get('is_featured') else 0
    is_popular = 1 if request.form.get('is_popular') else 0
    is_new = 1 if request.form.get('is_new') else 0

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('''
        UPDATE products
        SET name = ?, category = ?, price = ?, stock = ?,
            description = ?, image_url = ?, is_featured = ?,
            is_popular = ?, is_new = ?
        WHERE id = ?
```

```
    ''', (name, category, price, stock, description, image_url, is_featured, is_popular, is_new,
product_id))
    conn.commit()
    conn.close()

    flash("Product updated successfully!", "success")
    return redirect(url_for('admin_dashboard'))

@app.route('/admin/products/delete/<int:product_id>', methods=['POST'])
@admin_required
def admin_delete_product(product_id):
    verify_csrf()
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM products WHERE id = ?", (product_id,))
    conn.commit()
    conn.close()

    flash("Product deleted successfully.", "info")
    return redirect(url_for('admin_dashboard'))

@app.route('/admin/orders/status/<int:order_id>', methods=['POST'])
@admin_required
def admin_update_order_status(order_id):
    verify_csrf()
    new_status = request.form.get('status')
    valid_statuses = ['Processing', 'Shipped', 'Delivered', 'Cancelled']

    if new_status in valid_statuses:
        conn = get_db()
        cursor = conn.cursor()
```

```
        cursor.execute("UPDATE orders SET status = ? WHERE id = ?", (new_status, order_id))
        conn.commit()
        conn.close()
        flash(f"Order status updated to '{new_status}'.", "success")
    else:
        flash("Invalid order status.", "danger")

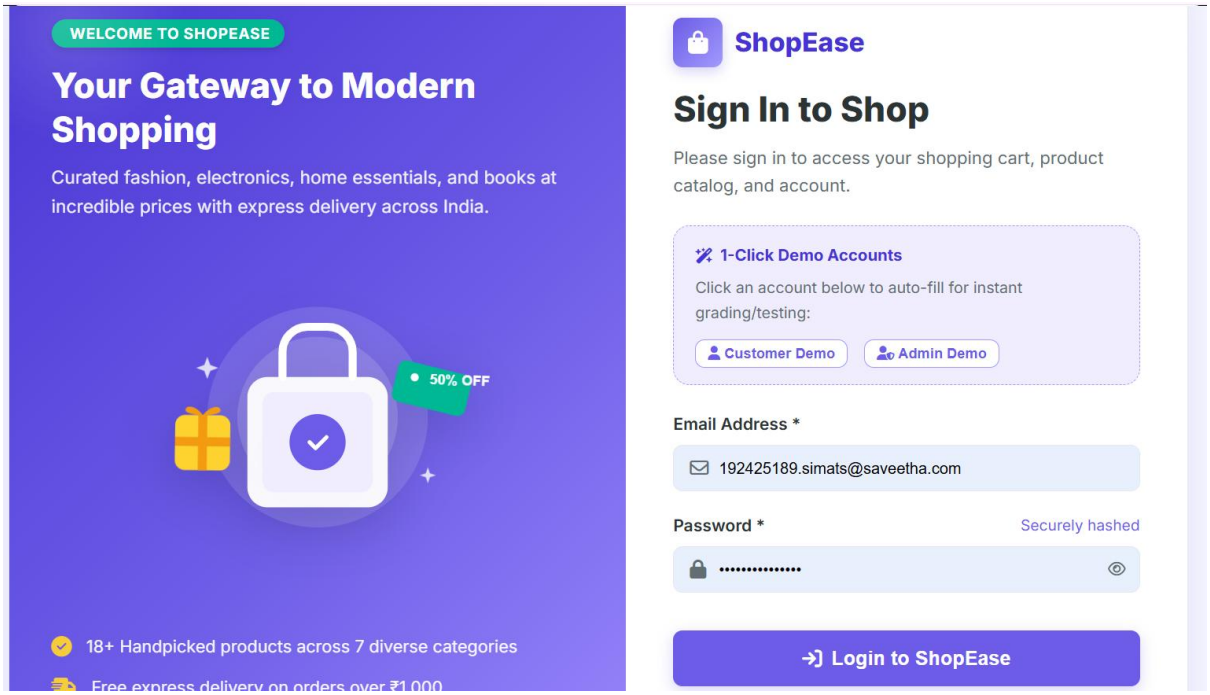
    return redirect(url_for('admin_dashboard'))

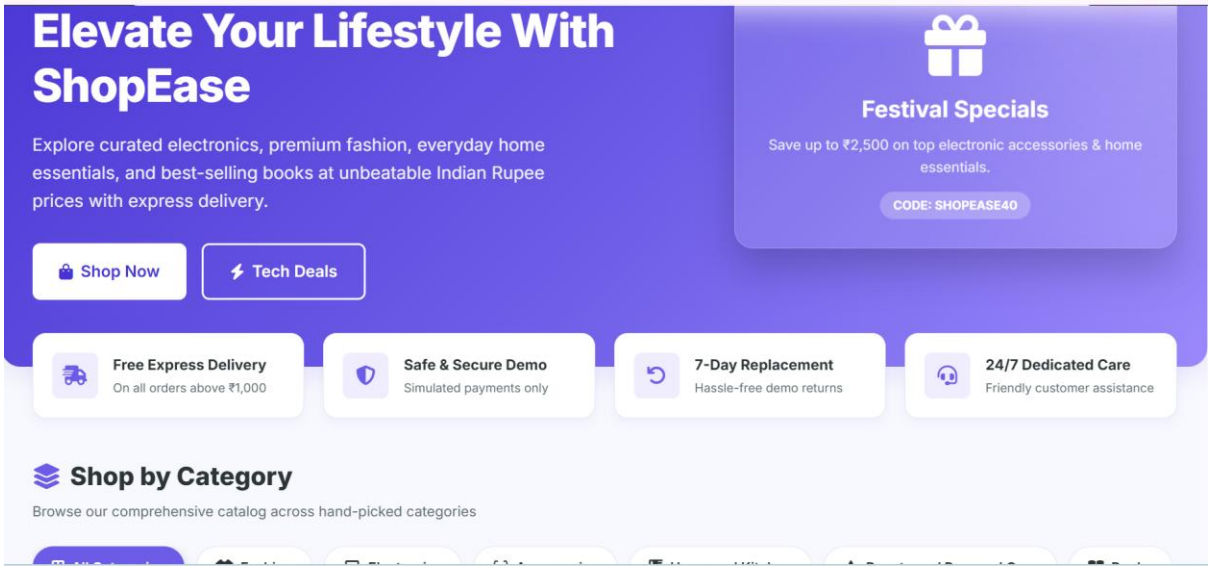
# ------------------------------------------------------------
# Error Handlers
# ------------------------------------------------------------
@app.errorhandler(404)
def page_not_found(e):
    return render_template('404.html', message=str(e)), 404

@app.errorhandler(500)
def internal_server_error(e):
    return render_template('404.html', message="An unexpected server error occurred."), 500

if __name__ == '__main__':
    app.run(debug=True, port=5000, use_reloader=False)
```

OUTPUT:


WELCOME TO SHOPEASE
Your Gateway to Modern Shopping
Curated fashion, electronics, home essentials, and books at incredible prices with express delivery across India.
50% OFF
18+ Handpicked products across 7 diverse categories
ShopEase
Sign In to Shop
Please sign in to access your shopping cart, product catalog, and account.
1-Click Demo Accounts
Click an account below to auto-fill for instant grading/testing:
Customer Demo
Admin Demo
Email Address *
192425189.simats@saveetha.com
Password *
Securely hashed
Login to ShopEase


Elevate Your Lifestyle With ShopEase
Explore curated electronics, premium fashion, everyday home essentials, and best-selling books at unbeatable Indian Rupee prices with express delivery.
Shop Now
Tech Deals
Festival Specials
Save up to ₹2,500 on top electronic accessories & home essentials.
CODE: SHOPEASE40
Free Express Delivery
On all orders above ₹1,000
Safe & Secure Demo
Simulated payments only
7-Day Replacement
Hassle-free demo returns
24/7 Dedicated Care
Friendly customer assistance
Shop by Category
Browse our comprehensive catalog across hand-picked categories

Fashion

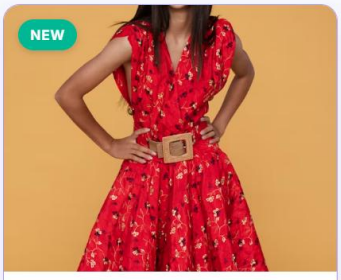


FASHION

Unisex Vintage Washed Denim Jacket

★★★★½ (89)

₹2,799.00 Add

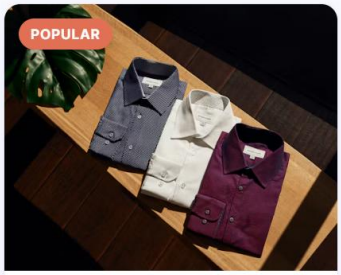


FASHION

Women's Lavender Floral A-Line Dress

★★★★½ (210)

₹2,199.00 Add



FASHION

Men's Classic Slim-Fit Oxford Shirt

★★★★½ (142)

₹1,299.00 Add

Your Shopping Cart

← Continue Shopping

PRODUCT	PRICE	QUANTITY	TOTAL	ACTION
Women's Lavender Floral A-Line Dress Category: Fashion	₹2,199.00	- 1 +	₹2,199.00	

Clear Cart + Add More Items

Order Summary

Items Subtotal (1 items) ₹2,199.00

Standard Delivery FREE

Congratulations! You've unlocked **FREE Delivery**.

Total Amount **₹2,199.00**

* All applicable taxes are included. Final shipping and discounts calculated at checkout.

Proceed to Checkout

Guaranteed Safe & Secure Demo Checkout

Complete Your Order

← Return to Cart

Shipping & Delivery Details

Full Name *
John Doe

Email Address *
john@example.com

Phone Number *
+91 98765 43210

Street Address / Flat / Building *
Flat 402, Lavender Heights, MG Road

City *
Bengaluru

State *
Karnataka

PIN / Postal Code *
560001

Order Review (1 items)

Women's Lavender Floral A-Line Dress
Qty: 1 × ₹2,199.00 ₹2,199.00

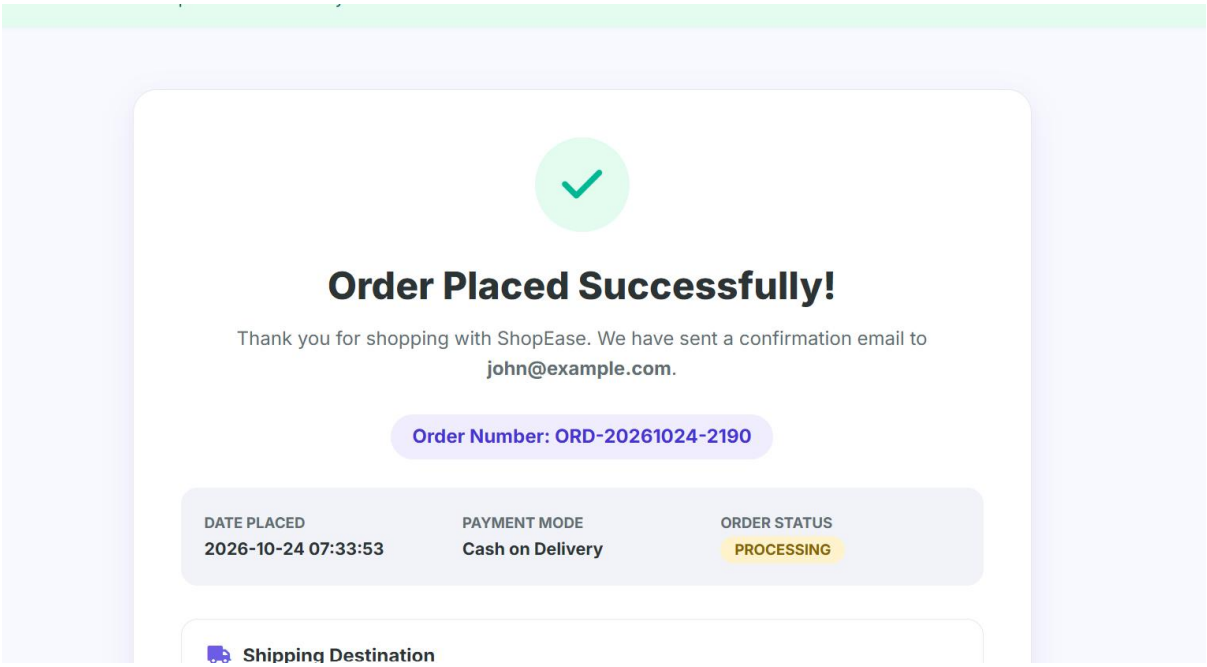
Subtotal ₹2,199.00

Shipping FREE

Total Payable **₹2,199.00**

Confirm & Place Demo Order

By confirming, you agree that this order is for testing & learning purposes only.



Order Placed Successfully!

Thank you for shopping with ShopEase. We have sent a confirmation email to **john@example.com**.

Order Number: ORD-20261024-2190

DATE PLACED	PAYMENT MODE	ORDER STATUS
2026-10-24 07:33:53	Cash on Delivery	PROCESSING

Shipping Destination